

Lesson 20: Inheritance - Syntax

Before beginning this lesson, you should have an understanding of the idea of inheritance. If you do not, please read [lesson 19](#). This lesson will consist of an overview of the syntax of inheritance, the use of the keywords `public`, `private`, and `protected`, and then an example program following to demonstrate each. The syntax to denote one class as inheriting from another is simple. It looks like the following: `class Bear : public Animal`, in place of simply the keyword `class` and then the class name. The `": public base_class_name"` is the essential syntax of inheritance; the function of this syntax is that the class will contain all public and protected variables of the base class. Do not confuse the idea of a derived class having access to data members of a base class and specific instances of the derived class possessing data. The data members - variables and functions - possessed by the derived class are specific to the type of class, not to each individual object of that type. So, two different Bear objects, while having the same member variables and functions, may have different information stored in their variables; furthermore, if there is a class Animal with an object, say object BigAnimal, of that type, and not of a more specific type inherited from that class, those two bears will not have access to the data within BigAnimal. They will simply possess variables and functions with the same name and of the same type. A quick example of inheritance:

```
class Animal
{
    public:
    int legs;
    int arms;
    int age;
    Animal();
    ~Animal();
    void eat();
    void sleep();
    void drink();
};
//The class Animal contains information and functions
//related to all animals (at least, all animals this lesson uses)
class Cat : public Animal
{
    public:
    int fur_color;
    void Purr();
    void fish();
    void Mark_territory();
};
//For those of you familiar with cats
//eat of the above operations is unique
//to your friendly furry friends
//(or enemies, as the case may be)
```

A discussion of the keywords `public`, `private`, and `protected` is useful when discussing inheritance. The three keywords are used to control access to functions and variables stored within a class.

public:

The most open level of *data hiding*, anything that is public is available to all derived classes of a base class, and the public variables and data for each object of both the base and derived class is accessible by code outside the class. Functions marked public are generally those the class uses to give information to and take information from the outside world; they are typically the interface with the class. The rest of

the class should be hidden from the user (This hidden nature and the highly focused nature of classes is known collectively as encapsulation). The syntax for public is:

`public:`

Everything following is public until the end of the class or another data hiding keyword is used.

protected:

Variables and functions marked protected are inherited by derived classes; however, these derived classes hide the data from code outside of any instance of the object. Keep in mind, even if you have another object of the same type as your first object, the second object cannot access a protected variable in the first object. Instead, the second object will have its own variable with the same name - but not necessarily the same data. Protected is a useful level of protection for important aspects to a class that must be passed on without allowing it to be accessed. The syntax is the same as that of public. Specifically,

`protected:`

private:

Private is the highest level of data-hiding. Not only are the functions and variables marked private not accessible by code outside the specific object in which that data appears, but private variables and functions are not inherited. The level of data protection afforded by protected is generally more flexible than that of the private level. Of course, there is a certain joy in protecting your data with the keyword private. The syntax remains the same.

`private:`